

K.R. MANGALAM UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
B.TECH CSE (AI & ML)

IMPLEMENTATION OF XOR, MUX AND DMUX CHIPS USING HDL
NAND2TETRIS HARDWARE SIMULATOR

Name	Divyansh Sharma
Roll No. / Enrollment No.	2501730385
Section	E
Program	B.Tech CSE (AI & ML)
University	K.R. Mangalam University

Objective:

To implement XOR, 2-way MUX, 4-way 16-bit MUX, and DMux chips using Nand2Tetris HDL, construct their truth tables, and verify the implementations using the corresponding Nand2Tetris test scripts.

1. XOR Gate

The XOR gate gives output 1 when exactly one of the two inputs is 1.

A	B	OUT
0	0	0
0	1	1
1	0	1
1	1	0

HDL implementation:

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Or(a=a, b=b, out=orAB);
    And(a=a, b=b, out=andAB);
    Not(in=andAB, out=notAnd);
    And(a=orAB, b=notAnd, out=out);
}
```

Test file: Xor.tst. The test script loads Xor.hdl, compares the output with Xor.cmp, and evaluates the input combinations.

The screenshot displays the NAND2Tetris Hardware Simulator interface. The main window shows the HDL code for a Demultiplexer (DMux) chip. The code defines the chip's inputs (in, sel), outputs (a, b), and internal logic using AND, OR, and NOT gates. The test script (Xor.tst) is loaded, and the simulation is successful, as indicated by the green status bar at the bottom.

Chip DMux

Input pins: in = 1, sel = 1

Output pins: a = 0, b = 1

Internal pins: notSel = 0

Test Script: Compare File, Output File, Diff Table

```
5
6 load DMux.hdl,
7 compare-to DMux.cmp,
8 output-list in sel a b;
9
10 set in 0,
11 set sel 0,
12 eval,
```

Simulation successful: The output file is identical to the compare file

Figure 1: XOR HDL implementation and test script evidence.

2. 2-Way Multiplexer (MUX)

A 2-way MUX selects one of two inputs. When sel = 0, OUT = A; when sel = 1, OUT = B.

A	B	SEL	OUT
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

HDL implementation:

```
CHIP Mux {
  IN a, b, sel;
  OUT out;

  PARTS:
    Not(in=sel, out=notSel);
    And(a=a, b=notSel, out=aPart);
    And(a=b, b=sel, out=bPart);
    Or(a=aPart, b=bPart, out=out);
}
```

Test file: Mux.tst. The test script loads Mux.hdl, compares the output with Mux.cmp, and evaluates the required combinations.

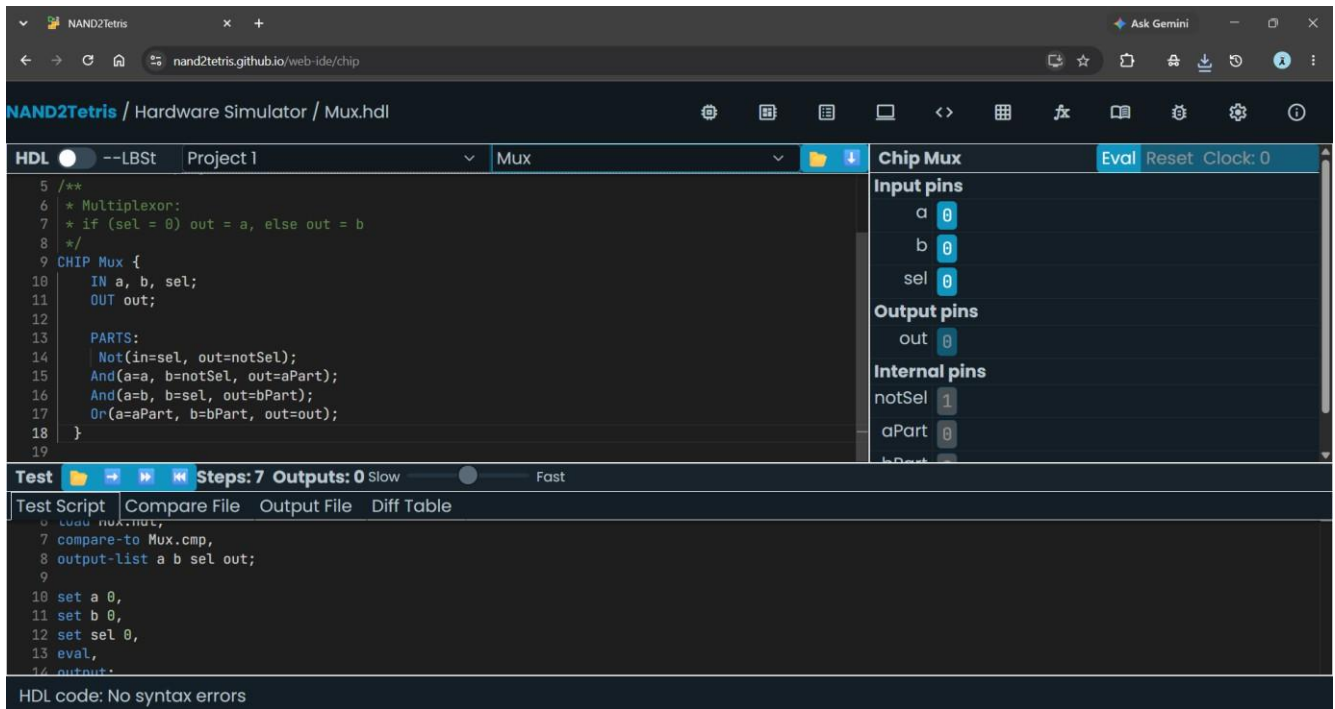


Figure 2: 2-way MUX HDL implementation and test script evidence.

3. 4-Way 16-Bit Multiplexer (Mux4Way16)

Mux4Way16 selects one of four 16-bit inputs using two select bits.

SEL[1]	SEL[0]	Selected Input
0	0	A
0	1	B
1	0	C
1	1	D

HDL implementation:

```

CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
        Mux16(a=a, b=b, sel=sel[0], out=ab);
        Mux16(a=c, b=d, sel=sel[0], out=cd);
        Mux16(a=ab, b=cd, sel=sel[1], out=out);
}

```

Test file: Mux4Way16.tst. The test script loads Mux4Way16.hdl, compares the output with Mux4Way16.cmp, and tests all four select combinations.

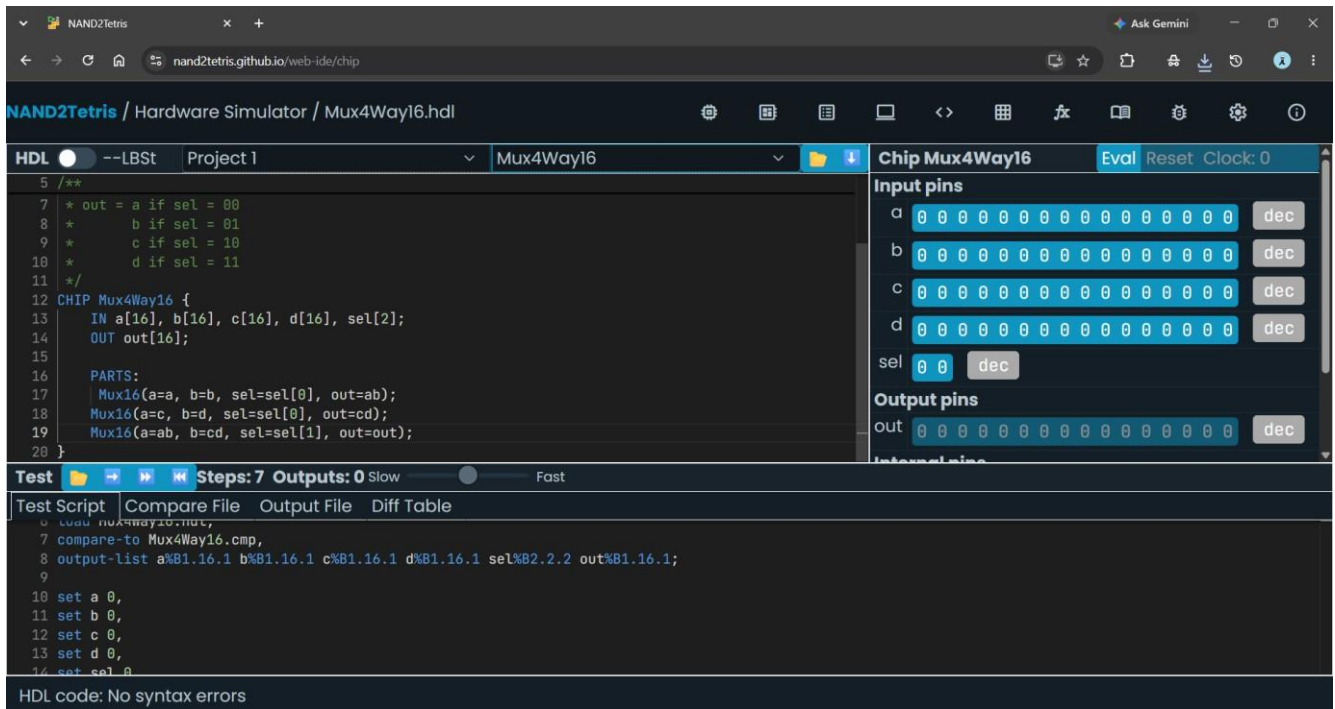


Figure 3: 4-way 16-bit MUX HDL implementation and test script evidence.

4. Demultiplexer (DMux)

The DMux routes one input to one of two outputs. When sel = 0, the input is routed to A; when sel = 1, it is routed to B.

IN	SEL	A	B
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

HDL implementation:

```

CHIP DMux {
  IN in, sel;
  OUT a, b;

  PARTS:
    Not(in=sel, out=notSel);
    And(a=in, b=notSel, out=a);
    And(a=in, b=sel, out=b);
}

```

Test file: DMux.tst. The test script loads DMux.hdl, compares the output with DMux.cmp, and verifies the expected results.

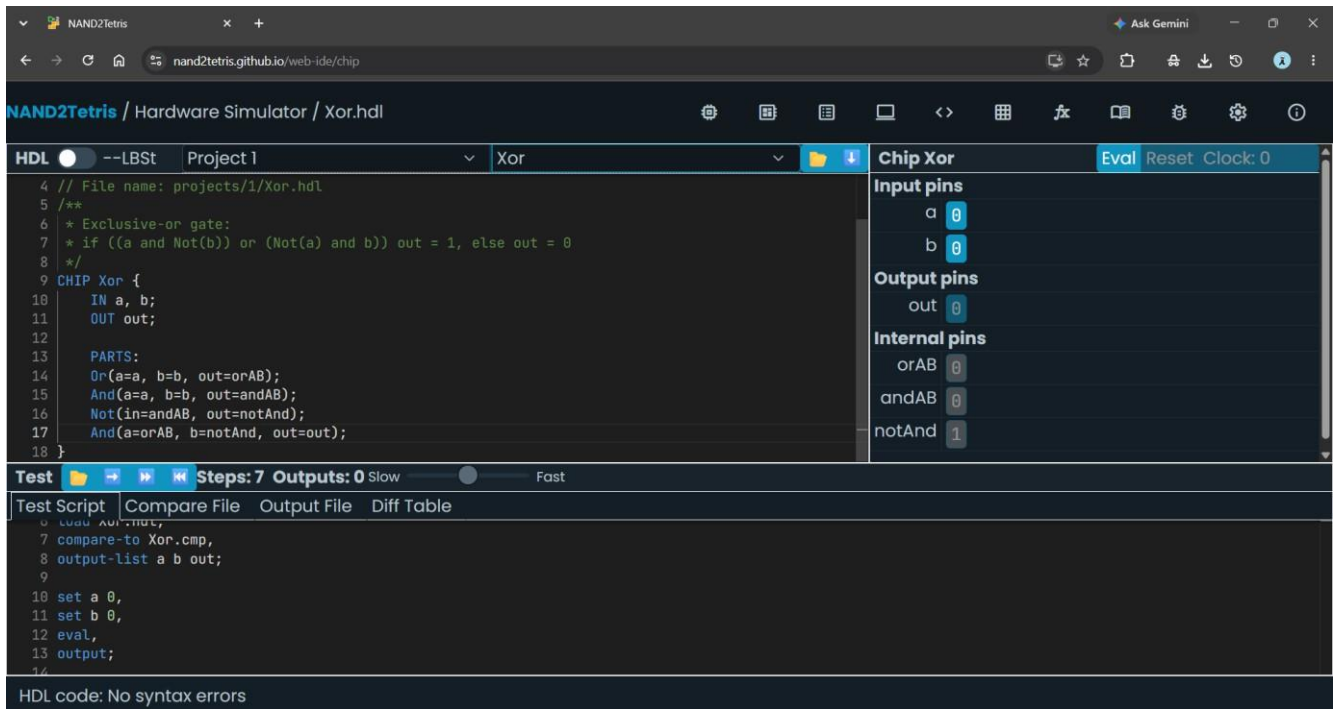


Figure 4: DMux HDL implementation and successful simulation evidence.

5. Verification Summary

Chip	HDL File	Test File	Evidence
XOR	Xor.hdl	Xor.tst	Simulator screenshot
2-way MUX	Mux.hdl	Mux.tst	Simulator screenshot
4-way 16-bit MUX	Mux4Way16.hdl	Mux4Way16.tst	Simulator screenshot
DMux	DMux.hdl	DMux.tst	Simulation successful

6. Conclusion

The XOR, 2-way MUX, 4-way 16-bit MUX, and DMux chips were implemented using Nand2Tetris HDL. Their truth tables, HDL implementations, test scripts, and simulator screenshots are included as evidence of implementation and verification.